

Abstraction means hiding implementation details and showing only essential features to the user.

In Java, abstraction is achieved using; 1) Abstract class
2) Interface

When to use Interface and when Abstract class?

⇒ Example of Bank Application.

→ In a bank system:

→ All payment methods must follow some rules

→ But some payment methods share common code.

Case 1: Interface

* An interface is like a rule book.

→ It tells what to do, Not how to do.

Exp: All payment modes must have pay().

But implementation differs.

Case 2: Abstract class:

⇒ An abstract class is like a base employee.

→ had some common behaviour

→ leave some work for subclasses.

Exp:

→ Bank provides common calculation

→ Leaves some work for subclasses

→ Child classes customize behavior

When to use interface

→ When multiple unrelated classes follow same rule

→ When we need multiple inheritance

→ When we want 100% abstraction

When to use abstract class

→ When classes are closely related

→ When we want shared code + abstract methods

→ When we want constructors and instance variables

ABSTRACT CLASS Example

```
public abstract class Test {
```

```
    public int a, b;
```

```
    public int getVal(int a, int b) {
```

```
        return a+b;
```

```
    public abstract String absmethod();
```

//child class

public class Newabs extends Test {

@Override

public String absMethod() {

return "This is new Abs class";

INTERFACE Example

public interface inter {

static final int a = 10;

default int m() {

return a;

}

Main.java

public class Main extends Newabs {

public static void main (String[] args) {

Test t = new Main();

System.out.println (t.getVal (12,13));

System.out.println (t.absMethod());

@Override

public String absMethod() { return "This is new

2) Are Interface Method calls slower than Abstract?

⇒ Not exactly. performance depends on:

- 1) Type of reference.
- 2) JVM optimization.

* calling a method via interface reference is slower.

* calling same method via class/abstract reference is faster.

Code

```
interface Shape {  
    int area();  
}
```

```
abstract class BaseShape implements Shape {  
    int side = 10;
```

```
    public int area() {
```

```
        return side * side;
```

```
    }
```

```
class Square extends BaseShape {};
```

```
Shape s1 = new Square(); // Interface
```

```
BaseShape s2 = new Square(); // Abstract
```

```
Square s3 = new Square(); // Concrete
```

here.

Fastest $\rightarrow$ square reference

Slowest $\rightarrow$ Interface reference

Because

$\rightarrow$ Interface calls require extra dynamic lookup.

$\rightarrow$ JVM still optimizes them.

$\rightarrow$ Difference is negligible in real applications.

#ABSTRACT vs INTERFACE

Features	Abstract Class	Interface
Methods	Abstract + Concrete	Abstract + Default.
Variables	Instance variables	public static final only
Inheritance	Single	multiple
Constructor	Allowed	Not Allowed
Access Modifier	Any	public only
Implementation	Partial	mostly none
Use Case	Base class	Rule/Contract
Speed	Slightly faster	Slightly Slower.